



Worse Parent Heuristic F = 24.30

```
def heuristics(edge_attr: np.ndarray) -> np.ndarray:
    ...
    heuristic_scores = np.exp(normalized_attr) / np.exp(1.0)
    return heuristic_scores.ravel()
```

Better Parent Heuristic F = 10.71

```
def heuristics(edge_attr: np.ndarray) -> np.ndarray:
    ...
    inverse_priority = 1 - normalized_attr
    epsilon = 1e-5
    heuristics_output = inverse_priority + epsilon
    return heuristic_output
```



Short-term Reflection

The better performance of the inverse scoring method suggests lower attribute values indicate more promising edges. **Prioritize edges with lower attributes** after normalization.



Offspring Heuristic F = 6.42

```
def heuristics(edge_attr: np.ndarray) -> np.ndarray:
    epsilon = 1e-9
    norm_attrs = (edge_attr - np.min(edge_attr)) /
        (np.max(edge_attr) - np.min(edge_attr) + epsilon)
    # Invert the normalized attributes
    promise_scores = 1 / (norm_attrs.flatten() + epsilon)
    return promise_scores
```

Crossover with Short-term Reflection



Elite Heuristic F = 6.36

```
def heuristics(edge_attr: np.ndarray) -> np.ndarray:
    epsilon = 1e-9
    norm_attrs = ...
    ...
    promise_scores = np.exp(-adaptive_norm_attrs)
    return promise_scores.flatten()
```



Long-term Reflection

Focus on adapting heuristics dynamically with real-time data analysis, emphasizing statistical distributions (**skewness**, **kurtosis**) and robust statistics (**median**, **MAD**) for edge attribute transformation.



Mutated Heuristic F = 5.83

```
def heuristics(edge_attr: np.ndarray) -> np.ndarray:
    epsilon = 1e-9
    # Median and MAD for initial data understanding
    median_val = np.median(edge_attr)
    mad = ...
    # Compute skewness and kurtosis using robust method
    skewness = ...
    kurtosis = ...
    norm_attrs = (edge_attr - median_val) / mad
    transformed_attrs = np.exp(
        -norm_attrs * (skewness * np.sqrt(kurtosis))
    )
    return transformed_attrs.flatten()
```

Mutation with Long-term Reflection